

6.4 A better API for state actions

Looking back at our implementations, we'll notice a common pattern: each of our functions has a type of the form `RNG => (A, RNG)` for some type `A`. Functions of this type are called *state actions* or *state transitions* because they transform `RNG` states from one to the next. These state actions can be combined using *combinators*, which are higher-order functions that we'll define in this section. Since it's pretty tedious and repetitive to pass the state along ourselves, we want our combinators to pass the state from one action to the next automatically.

To make the type of actions convenient to talk about, and to simplify our thinking about them, let's make a type alias for the `RNG` state action data type:

```
type Rand[+A] = RNG => (A, RNG)
```

We can think of a value of type `Rand[A]` as “a randomly generated `A`,” although that's not really precise. It's really a state action—a *program* that depends on some `RNG`, uses it to generate an `A`, and also transitions the `RNG` to a new state that can be used by another action later.

We can now turn methods such as `RNG's nextInt` into values of this new type:

```
val int: Rand[Int] = _.nextInt
```

We want to write combinators that let us combine `Rand` actions while avoiding explicitly passing along the `RNG` state. We'll end up with a kind of domain-specific language that does all of the passing for us. For example, a simple `RNG` state transition is the `unit` action, which passes the `RNG` state through without using it, always returning a constant value rather than a random value:

```
def unit[A](a: A): Rand[A] =  
  rng => (a, rng)
```

There's also `map` for transforming the output of a state action without modifying the state itself. Remember, `Rand[A]` is just a type alias for a function type `RNG => (A, RNG)`, so this is just a kind of function composition:

```
def map[A,B](s: Rand[A])(f: A => B): Rand[B] =  
  rng => {  
    val (a, rng2) = s(rng)  
    (f(a), rng2)  
  }
```

As an example of how `map` is used, here's `nonNegativeEven`, which reuses `nonNegativeInt` to generate an `Int` that's greater than or equal to zero and divisible by two:

```
def nonNegativeEven: Rand[Int] =  
  map(nonNegativeInt)(i => i - i % 2)
```